AEROSTACK2

# 3-Drone Simulation

*ROS 2 Humble and Gazebo Based Multi-Drone Formation Control*

| Item | Details |
|---|---|
| Framework | AeroStack2 |
| Middleware | ROS 2 Humble |
| Simulator | Gazebo |
| Operating System | Ubuntu 22.04 |
| Technology Stack | Python 3, ROS 2 (rclpy), AeroStack2 DroneInterface, Gazebo simulation |

## 2. Executive Summary

The system uses one leader drone (drone0) and two follower drones (drone1, drone2). The leader is commanded along a predefined five-waypoint route; follower targets are computed at each waypoint by applying fixed (x, y, z) offsets to the leader's planned position, producing a triangular (delta) formation. Arming and Offboard activation are performed sequentially, one drone at a time; takeoff, formation GoTo commands and landing are then dispatched concurrently using one Python thread per drone so that all three drones begin moving toward their respective targets at the same time.

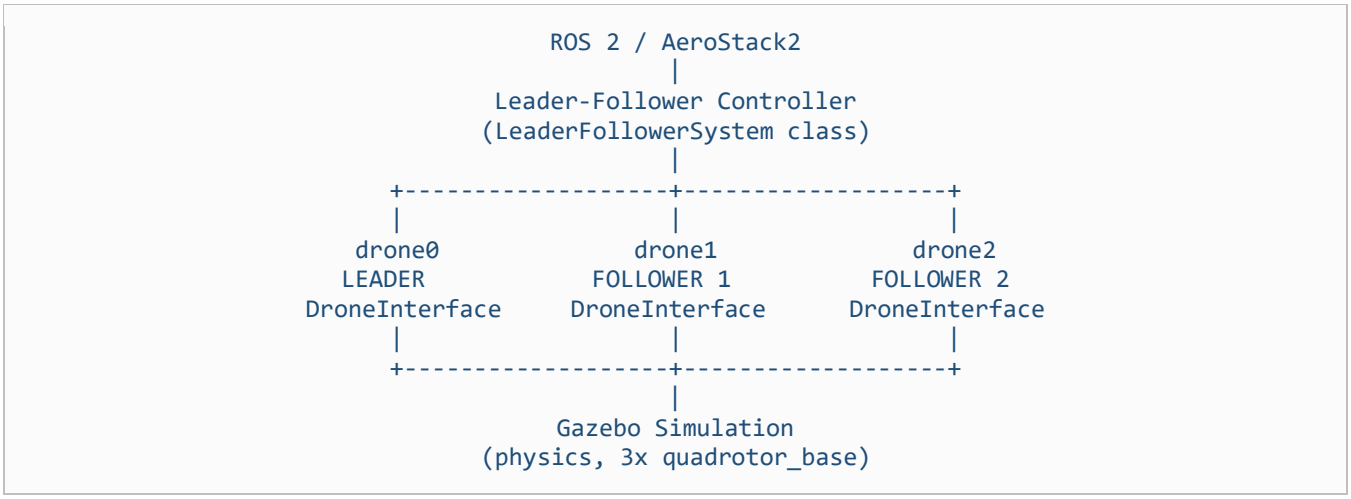
The final validated run completed the full mission lifecycle: ROS 2 interface initialization, arming, Offboard activation, concurrent takeoff to 1.0 m, five formation waypoints (all returning SUCCESS for all three drones), return to the initial waypoint, coordinated landing, and clean interface shutdown. The mission script printed “FORMATION MISSION COMPLETED SUCCESSFULLY” after the final waypoint, and this was confirmed independently by a post-mission /platform/info check on all three drones.

## 3. System Overview

The system is built entirely on the AeroStack2 framework running over ROS 2 Humble, with Gazebo providing the physics simulation for three quadrotor models. Each simulated drone is exposed to the mission script as an independent ROS 2 namespace (/drone0, /drone1, /drone2), and each namespace is wrapped by an AeroStack2 DroneInterface object in Python.

The mission logic sits above the DroneInterface layer. A LeaderFollowerSystem class owns all three Drone objects, designates drone0 as the leader, and computes follower targets from fixed relative offsets. Arming and Offboard activation are performed sequentially for each drone, while takeoff, formation GoTo commands and landing are dispatched concurrently using one Python thread per drone.

### *System Architecture*



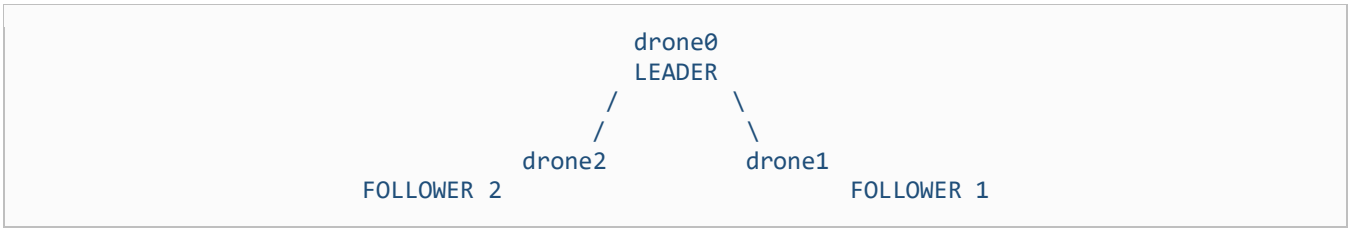

Each DroneInterface exposes the AeroStack2 behaviors used in this project: arm(), offboard(), takeoff(), go_to.go_to(), and land(). The GoTo action is backed by the as2_msgs/action/GoToWaypoint action server, which reports feedback such as actual speed and actual distance-to-goal while the drone is in transit.

## 4. Software and Simulation Environment

| Item | Configuration |
|---|---|
| Operating System | Ubuntu 22.04 |
| ROS | ROS 2 Humble |
| Simulator | Gazebo |
| Framework | AeroStack2 |
| AeroStack2 workspace | ~/aerostack2_ws |
| Project root | ~/project_gazebo |
| Main mission script | leader_follower_waypoint_backup.py |
| World configuration | config/world_swarm.yaml |
| Drone model | quadrotor_base |

*Table 1. Software and simulation environment configuration.*

## 5. Three-Drone Configuration




## 6. Formation Control Strategy

Given a leader target position L = (Lx, Ly, Lz), the follower targets are computed by adding a fixed offset to the leader's target:

```
F1 = (Lx + 1.5, Ly - 1.5, Lz)      # Follower 1 (drone1)
F2 = (Lx - 1.5, Ly - 1.5, Lz)      # Follower 2 (drone2)
```

| Follower | Drone | Offset (x, y, z) m |
|---|---|---|
| Follower 1 | drone1 | (+1.5, -1.5, 0.0) |
| Follower 2 | drone2 | (-1.5, -1.5, 0.0) |

*Table 3. Fixed relative formation offsets used to compute follower targets.*

Because both followers are offset behind and to either side of the leader by the same magnitude in y, and split symmetrically in x, the three drones maintain a consistent triangular (delta) formation as the leader advances through each waypoint.

The leader follows a predefined route of five waypoints; the follower targets are recalculated once per waypoint, from the leader's planned (not measured) position at that waypoint. This is explicitly a fixed-offset,

waypoint-synchronized formation strategy — not a continuous controller that reads the leader's live pose and reacts to it in real time.

## 7. Mission Waypoint Design

| WP | Leader | Follower 1 | Follower 2 |
|---|---|---|---|
| 1 | (0.00, 0.00, 1.00) | (1.50, -1.50, 1.00) | (-1.50, -1.50, 1.00) |
| 2 | (2.00, 0.00, 1.00) | (3.50, -1.50, 1.00) | (0.50, -1.50, 1.00) |
| 3 | (2.00, 2.00, 1.00) | (3.50, 0.50, 1.00) | (0.50, 0.50, 1.00) |
| 4 | (0.00, 2.00, 1.00) | (1.50, 0.50, 1.00) | (-1.50, 0.50, 1.00) |
| 5 | (0.00, 0.00, 1.00) | (1.50, -1.50, 1.00) | (-1.50, -1.50, 1.00) |

*Table 4. Five-waypoint mission: leader and derived follower targets.*

The leader follows a closed, square-shaped route at 1.0 m altitude:

```
(0,0)  ->  (2,0)  ->  (2,2)  ->  (0,2)  ->  (0,0)
```

while the two followers maintain the same relative offset at every point along the route, so the formation traces a congruent triangular path alongside the leader.

## 8. Mission Parameters

| Parameter | Value |
|---|---|
| Takeoff height | 1.0 m |
| Takeoff speed | 0.7 m/s |
| Mission speed | 0.5 m/s |
| Landing speed | 0.4 m/s |
| Formation stabilization time | 2.0 s |
| Follower 1 offset | (+1.5, -1.5, 0.0) m |
| Follower 2 offset | (-1.5, -1.5, 0.0) m |

*Table 5. Mission parameters used in the final validated run.*

## 9. Project Structure

```
~/project_gazebo/
├── launch_as2.bash
├── launch_ground_station.bash
├── stop.bash
├── leader_follower_waypoint_backup.py
├── leader_follower.py
├── mission_swarm.py
└── config/
```

```
    └── world_swarm.yaml
```

| File | Purpose |
|---|---|
| launch_as2.bash | Launches AeroStack2 and Gazebo for the configured drone namespaces (-m enables multi-drone mode). |
| launch_ground_station.bash | Optional ground-station viewer for monitoring drone state during a run. |
| stop.bash | Shuts down the simulation and associated AeroStack2 processes. |
| leader_follower_waypoint_backup.py | Main mission script used for the successful run documented in this report. |
| leader_follower.py | Earlier leader–follower implementation developed during the project. |
| mission_swarm.py | Geometric swarm-choreography script (delta/line formations) developed alongside the leader–follower approach. |
| config/world_swarm.yaml | Gazebo world definition for the three-drone swarm configuration. |

## 10. Environment Setup

The correct AeroStack2 workspace for this project is ~/aerostack2_ws. The following commands were used to source the environment before every run:

```
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
cd ~/project_gazebo
```

Only ~/aerostack2_ws/install/setup.bash provides the correct AeroStack2 overlay for this project; it is the workspace referenced throughout this report.

## 11. Simulation Launch

Launch the three-drone simulation (multi-drone mode):

```
./launch_as2.bash -m
```

This starts Gazebo with the swarm world configuration and brings up the AeroStack2 stack (platform, controller, state estimator and behavior servers) for each of the three drone namespaces.

Optional ground station, for live monitoring during a run:

```
./launch_ground_station.bash -m -v
```

Stop the simulation and associated processes:

```
./stop.bash
```

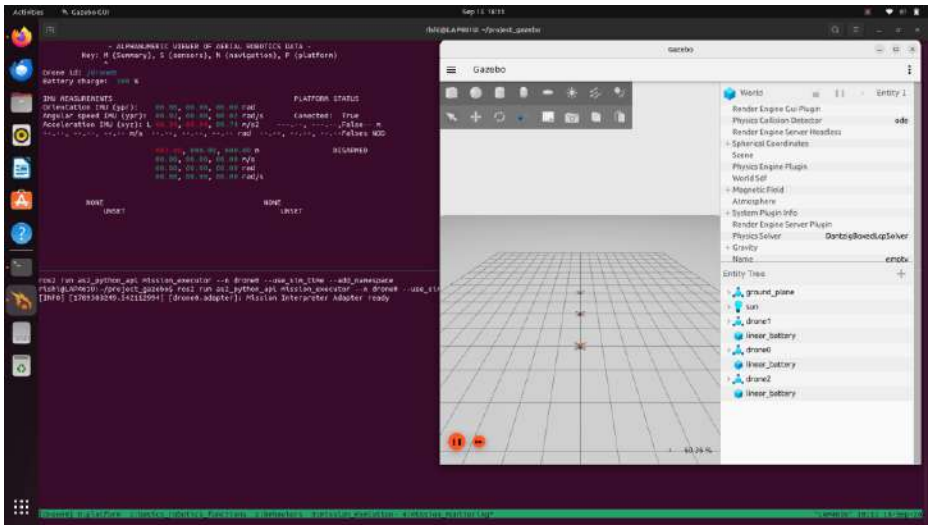


*Figure 3. AeroStack2 mission executor and alphanumeric drone-status viewer running alongside the Gazebo three-drone simulation.*

## 12. Initial Platform Verification

Before mission preparation, the platform state of all three drones was checked directly from the ROS 2 command line:

```
for d in drone0 drone1 drone2; do
  echo "========== $d =========="
  ros2 topic echo /$d/platform/info --once
done
```

| Drone | Connected | Armed | Offboard |
|---|---|---|---|
| drone0 | true | false | false |
| drone1 | true | false | false |
| drone2 | true | false | false |

*Table 6. Initial /platform/info state, prior to arming.*

This confirms that all three AeroStack2 interfaces were live and connected to their respective simulated drones before any mission commands were issued.

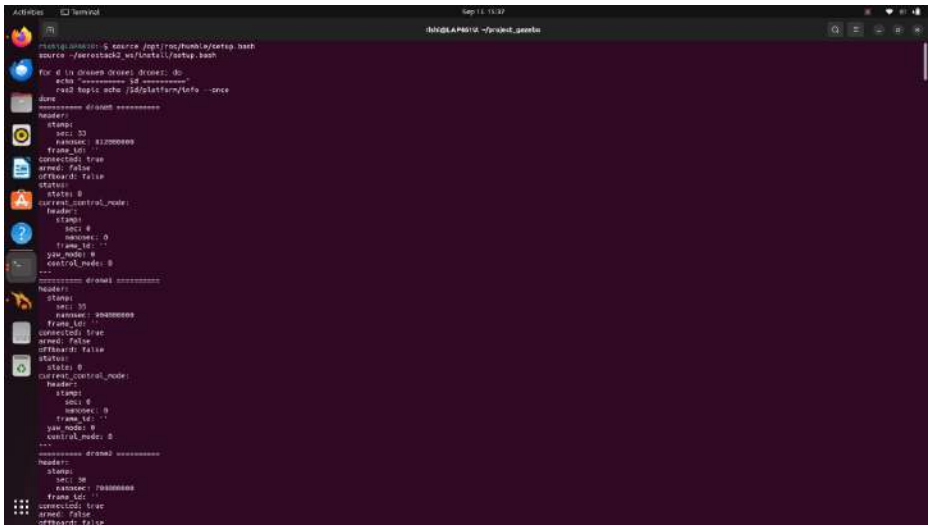


*Figure 4. Initial /platform/info verification showing all three drones connected, prior to arming.*

### *12.1 Supplementary ROS 2 Interface Verification*

In addition to the platform-info check, the underlying ROS 2 node graph and motion-reference topics were inspected directly to confirm that each drone exposed the expected control and state-estimation components.

```
ros2 node list | grep -E '^/drone(0|1|2)/' | grep -E \
  'controller|motion|platform|state_estimator|GoTo|Takeoff'
```

| Drone | Verified nodes |
|---|---|
| drone0 | GoToBehavior, TakeoffBehavior, controller_manager, platform, state_estimator |
| drone1 | GoToBehavior, TakeoffBehavior, controller_manager, platform, state_estimator |
| drone2 | GoToBehavior, TakeoffBehavior, controller_manager, platform, state_estimator |

*Table 7. ROS 2 nodes verified for each drone namespace.*

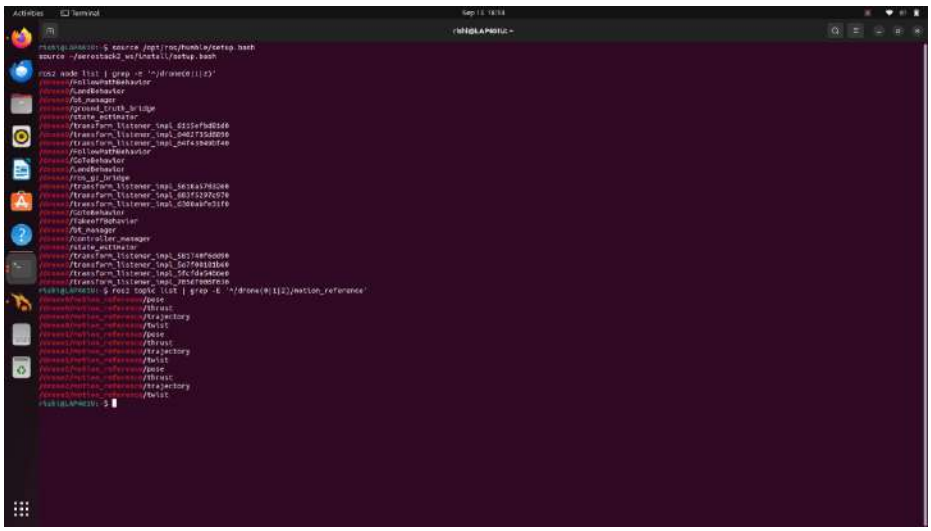


*Figure 5. ROS 2 node listing verifying GoToBehavior, TakeoffBehavior, controller_manager, platform and state_estimator nodes for all three drones.*

Motion-reference topics (pose, thrust, trajectory, twist) and self-localization pose data were also confirmed for all three drones, verifying that each drone was publishing usable state and reference data in the earth frame.

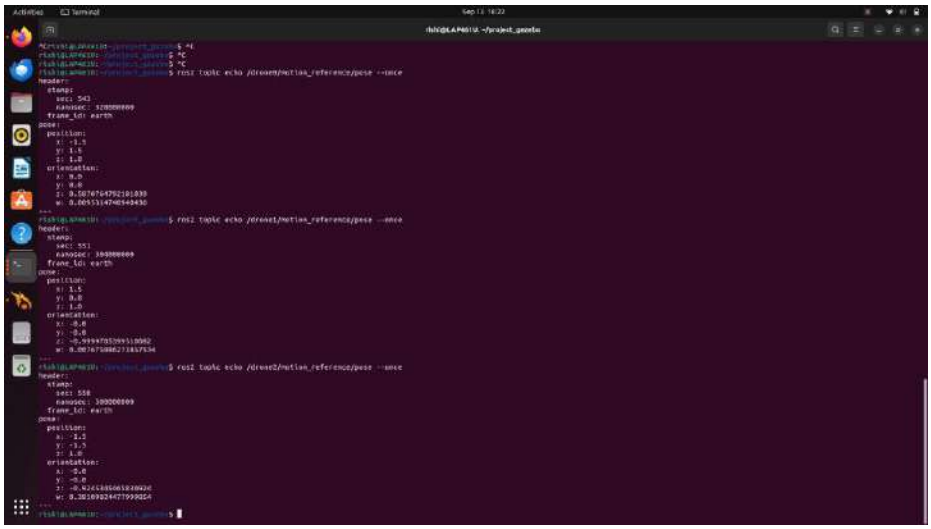


*Figure 6. Self-localization / motion-reference pose output confirming position and orientation data for drone0, drone1 and drone2 in the earth frame.*

## 13. Implementation Details

The mission logic is implemented as a small, self-contained architecture built on top of AeroStack2's DroneInterface:

| Component | Responsibility |
|---|---|
| Drone(DroneInterface) | Thin subclass that configures each drone with verbose logging and simulation time. |
| LeaderFollowerSystem | Owns all three Drone objects; assigns leader/follower roles; coordinates the full mission lifecycle. |
| prepare_drones() | Arms each drone and enables Offboard control in sequence: drone0, then drone1, then drone2. |
| takeoff() / takeoff_drone() | Issues concurrent takeoff commands to all three drones using one thread per drone. |
| move_formation() | Computes follower targets from the leader target and offsets, then dispatches concurrent GoTo commands. |
| execute_go_to() | Issues a single drone's GoTo command and records its result. |
| execute_mission() | Iterates over the five mission waypoints, calling move_formation() for each and reporting per-waypoint status. |
| land() / land_drone() | Issues concurrent landing commands to all three drones. |
| shutdown() | Cleanly shuts down each drone interface at the end of the mission or on error. |

Takeoff, GoTo and landing commands are all dispatched using Python's threading.Thread, with one thread per drone joined before the system proceeds to the next stage. This is used because the three drones need to begin moving toward their respective targets together rather than one after another — issuing the commands sequentially would introduce a timing offset between drones and break the intended formation geometry during transit.

## 14. Execution Procedure

Source ROS 2 Humble: source /opt/ros/humble/setup.bash

Source the AeroStack2 workspace: source ~/aerostack2_ws/install/setup.bash

Enter the project directory: cd ~/project_gazebo

Launch the Gazebo simulation: ./launch_as2.bash -m

Start the mission script: python3 leader_follower_waypoint_backup.py

Press ENTER once to begin the preparation stage; the script sequentially arms all three drones, enables Offboard control, and then commands concurrent takeoff to 1.0 m.

Press ENTER to start the leader–follower formation mission (five waypoints).

Observe the per-waypoint GoTo results and confirm FORMATION MISSION COMPLETED SUCCESSFULLY.

Press ENTER to land all three drones.

Run the final /platform/info verification loop to confirm the post-mission state.

This procedure is reproducible end-to-end by another engineer with access to the same workspace and project root.

## 15. Complete Working Source Code

The following is the complete, unmodified source code for leader_follower_waypoint_backup.py, used for the successful run documented in this report. It includes the corrected GoTo API call, concurrent takeoff/formation/landing via threading, the fixed formation offsets, the five mission waypoints, formation stabilization, and clean shutdown handling.

```
#!/usr/bin/env python3

import threading
import time

import rclpy

from aerostack2 import DroneInterface


TAKEOFF_HEIGHT = 1.0
TAKEOFF_SPEED = 0.7
LAND_SPEED = 0.4
MISSION_SPEED = 0.5

POSITION_TOLERANCE = 0.40
FORMATION_STABILIZATION_TIME = 2.0

FOLLOWER_1_OFFSET = (1.5, -1.5, 0.0)
FOLLOWER_2_OFFSET = (-1.5, -1.5, 0.0)

MISSION_WAYPOINTS = [
    (0.0, 0.0, 1.0),
    (2.0, 0.0, 1.0),
    (2.0, 2.0, 1.0),
    (0.0, 2.0, 1.0),
```

```
    (0.0, 0.0, 1.0),
]


class Drone(DroneInterface):
    def __init__(self, namespace):
        super().__init__(
            namespace,
            verbose=True,
            use_sim_time=True
        )


class LeaderFollowerSystem:
    def __init__(self):
        self.drones = {
            "drone0": Drone("drone0"),
            "drone1": Drone("drone1"),
            "drone2": Drone("drone2"),
        }

        self.leader = self.drones["drone0"]
        self.follower1 = self.drones["drone1"]
        self.follower2 = self.drones["drone2"]

        print("\n========================================")
        print("AEROSTACK2")
        print("3-DRONE LEADER-FOLLOWER SYSTEM")
        print("========================================")

        print("\nLeader   : drone0")
        print("Follower : drone1")
        print("Follower : drone2")

        print("\nFormation:")
        print("      drone0")
        print("     /      \\")
        print("  drone2    drone1")

        print("\n========================================")

    def prepare_drones(self):
        print("\n========================================")
        print("ARMING ALL DRONES")
        print("========================================")

        for name, drone in self.drones.items():
            print(f"{name}: arming...")
            result = drone.arm()
            print(f"{name}: arm result = {result}")

            print(f"{name}: enabling offboard...")
            result = drone.offboard()
            print(f"{name}: offboard result = {result}")

        print("\nAll drones prepared.")

    def takeoff_drone(self, name, drone, results):
        print(f"{name}: TAKEOFF -> {TAKEOFF_HEIGHT:.2f} m")
```

```
        result = drone.takeoff(
            TAKEOFF_HEIGHT,
            TAKEOFF_SPEED,
            False
        )
        results[name] = result
        print(f"{name}: takeoff command completed -> {result}")

    def takeoff(self):
        print("\n======================================")
        print("TAKEOFF")
        print("======================================")

        results = {}
        threads = []

        for name, drone in self.drones.items():
            thread = threading.Thread(
                target=self.takeoff_drone,
                args=(name, drone, results)
            )
            threads.append(thread)
            thread.start()

        for thread in threads:
            thread.join()

        print("\n======================================")
        print("TAKEOFF RESULTS")
        print("======================================")

        for name in self.drones:
            status = "SUCCESS" if results.get(name) else "FAILED"
            print(f"{name}: {status}")

        if all(results.get(name) for name in self.drones):
            print("\nALL DRONES REACHED TAKEOFF ALTITUDE.")
        else:
            print("\nWARNING: ONE OR MORE TAKEOFF COMMANDS FAILED.")

    def execute_go_to(self, name, drone, position, results):
        x, y, z = position

        print(f"{name}: GoTo -> ({x:.2f}, {y:.2f}, {z:.2f})")

        result = drone.go_to.go_to(
            x,
            y,
            z,
            MISSION_SPEED,
            "earth"
        )

        results[name] = result
        print(f"{name}: GoTo result = {result}")

    def move_formation(self, leader_position):
        lx, ly, lz = leader_position

        follower1_position = (
```

```
            lx + FOLLOWER_1_OFFSET[0],
            ly + FOLLOWER_1_OFFSET[1],
            lz + FOLLOWER_1_OFFSET[2]
        )

        follower2_position = (
            lx + FOLLOWER_2_OFFSET[0],
            ly + FOLLOWER_2_OFFSET[1],
            lz + FOLLOWER_2_OFFSET[2]
        )

        targets = {
            "drone0": leader_position,
            "drone1": follower1_position,
            "drone2": follower2_position,
        }

        print("\n--------------------------------------------------")
        print("FORMATION UPDATE")
        print("--------------------------------------------------")

        print(f"Leader     -> ({leader_position[0]:.2f}, {leader_position[1]:.2f},
{leader_position[2]:.2f})")
        print(f"Follower 1 -> ({follower1_position[0]:.2f}, {follower1_position[1]:.2f},
{follower1_position[2]:.2f})")
        print(f"Follower 2 -> ({follower2_position[0]:.2f}, {follower2_position[1]:.2f},
{follower2_position[2]:.2f})")

        print("\nStarting all three GoTo commands...")

        results = {}
        threads = []

        for name, drone in self.drones.items():
            thread = threading.Thread(
                target=self.execute_go_to,
                args=(name, drone, targets[name], results)
            )
            threads.append(thread)
            thread.start()

        for thread in threads:
            thread.join()

        print("\n--------------------------------------------------")
        print("WAYPOINT RESULTS")
        print("--------------------------------------------------")

        for name in self.drones:
            status = "SUCCESS" if results.get(name) else "FAILED"
            print(f"{name}: {status}")

        print(f"\nStabilizing formation for {FORMATION_STABILIZATION_TIME:.1f} seconds...")
        time.sleep(FORMATION_STABILIZATION_TIME)

        return all(results.get(name) for name in self.drones)

    def execute_mission(self):
        print("\n========================================")
        print("LEADER-FOLLOWER FORMATION MISSION")
```

```
        print("========================================")

        for index, waypoint in enumerate(MISSION_WAYPOINTS, start=1):
            print("\n========================================")
            print(f"WAYPOINT {index}/{len(MISSION_WAYPOINTS)}")
            print("========================================")

            print(f"Leader target = ({waypoint[0]:.2f}, {waypoint[1]:.2f}, {waypoint[2]:.2f})")

            success = self.move_formation(waypoint)

            if success:
                print(f"\nWaypoint {index} completed successfully.")
            else:
                print(f"\nWaypoint {index} had one or more failures.")

        print("\n========================================")
        print("FORMATION MISSION COMPLETED SUCCESSFULLY")
        print("========================================")

    def land_drone(self, name, drone, results):
        print(f"{name}: LANDING...")
        result = drone.land(
            LAND_SPEED,
            False
        )
        results[name] = result
        print(f"{name}: landing command completed -> {result}")

    def land(self):
        print("\n========================================")
        print("LANDING ALL DRONES")
        print("========================================")

        results = {}
        threads = []

        for name, drone in self.drones.items():
            thread = threading.Thread(
                target=self.land_drone,
                args=(name, drone, results)
            )
            threads.append(thread)
            thread.start()

        for thread in threads:
            thread.join()

        print("\n========================================")
        print("LANDING RESULTS")
        print("========================================")

        for name in self.drones:
            status = "SUCCESS" if results.get(name) else "FAILED"
            print(f"{name}: {status}")

        print("\nAll landing operations completed.")

    def shutdown(self):
```

```
        print("\n======================================")
        print("SHUTTING DOWN DRONE INTERFACES")
        print("======================================")

        for drone in self.drones.values():
            try:
                drone.shutdown()
            except Exception:
                pass


def main():
    rclpy.init()

    swarm = LeaderFollowerSystem()

    try:
        input("\nPress ENTER to ARM and TAKEOFF...")

        swarm.prepare_drones()

        swarm.takeoff()

        input("\nPress ENTER to start LEADER-FOLLOWER FORMATION...")

        swarm.execute_mission()

        input("\nPress ENTER to LAND all drones...")

        swarm.land()

    except KeyboardInterrupt:
        print("\nMission interrupted by user.")

    except Exception as e:
        print(f"\nMission error: {e}")

    finally:
        swarm.shutdown()

        if rclpy.ok():
            rclpy.shutdown()


if __name__ == "__main__":
    main()
```

Module summary: Drone wraps DroneInterface for a given namespace. LeaderFollowerSystem holds all three Drone instances and drives the mission through prepare_drones() → takeoff() → execute_mission() → land() → shutdown(), with move_formation() and execute_go_to() implementing the per-waypoint concurrent GoTo dispatch described in Section 14.

## 16. Arming and Offboard Results

```
drone0: arm result = True
drone0: offboard result = True
```

```
drone1: arm result = True
drone1: offboard result = True
drone2: arm result = True
drone2: offboard result = True

All drones prepared.
```

All three drones were armed and switched into Offboard control mode without error, confirming that each DroneInterface was correctly connected to its underlying platform before flight commands were issued.

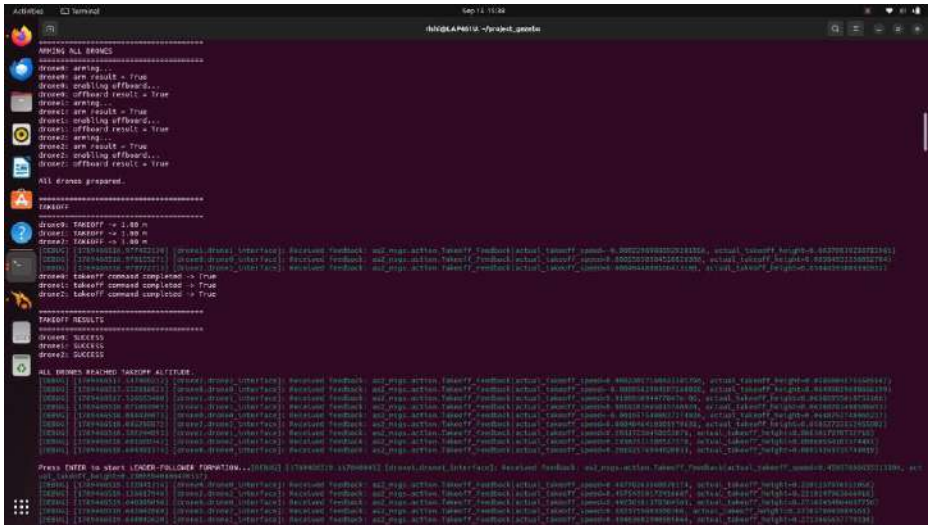


*Figure 7. Successful arming and Offboard activation for all three drones.*

## 17. Takeoff Results

```
TAKEOFF RESULTS
drone0: SUCCESS
drone1: SUCCESS
drone2: SUCCESS

ALL DRONES REACHED TAKEOFF ALTITUDE.
```

All three concurrent takeoff commands completed successfully, with every drone reaching the 1.0 m mission altitude before the formation stage began.

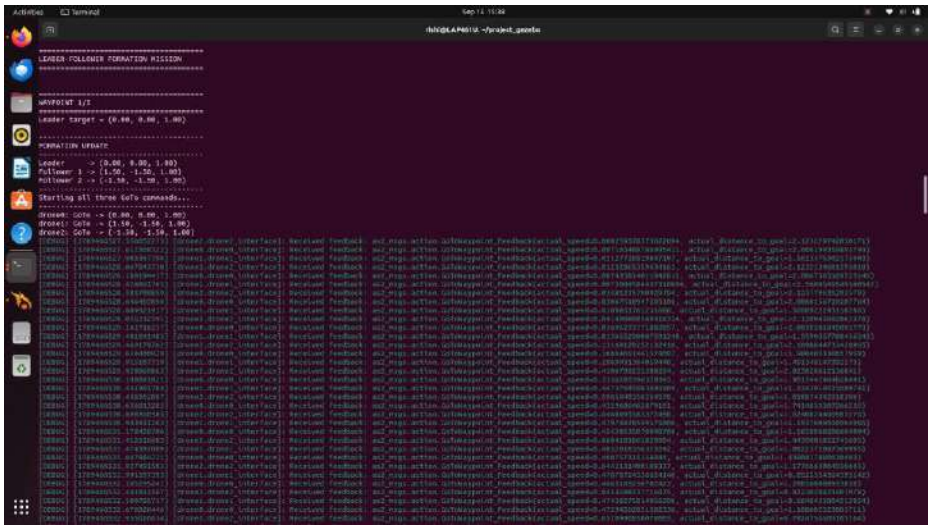


*Figure 8. Successful concurrent takeoff to the 1.0 m mission altitude.*

## 18. Formation Mission Results

| Waypoint | drone0 | drone1 | drone2 | Formation status |
|---|---|---|---|---|
| WP1 | SUCCESS | SUCCESS | SUCCESS | Completed |
| WP2 | SUCCESS | SUCCESS | SUCCESS | Completed |
| WP3 | SUCCESS | SUCCESS | SUCCESS | Completed |
| WP4 | SUCCESS | SUCCESS | SUCCESS | Completed |
| WP5 | SUCCESS | SUCCESS | SUCCESS | Completed |

*Table 8. Formation mission results — all three drones returned SUCCESS at every waypoint.*

All three drones returned a successful GoTo result at every one of the five waypoints, with no failed or retried commands recorded during the run.

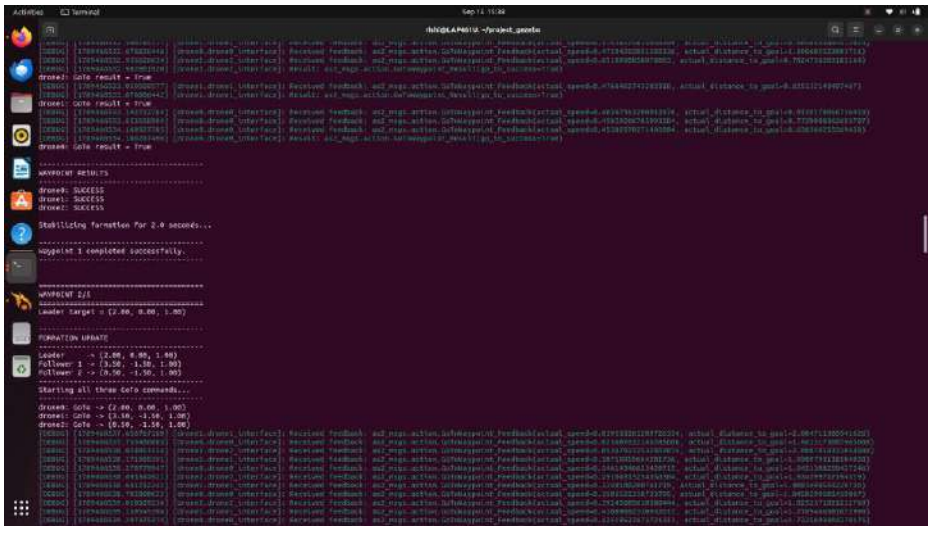


*Figure 9. Formation waypoint target calculation and concurrent GoTo command dispatch to all three drones.*

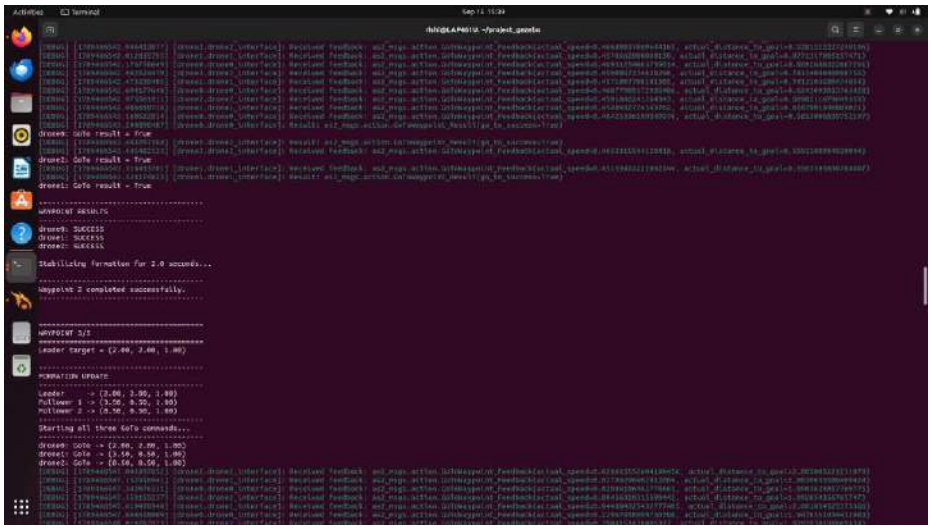


*Figure 10. Waypoint completion and transition to the next formation waypoint.*

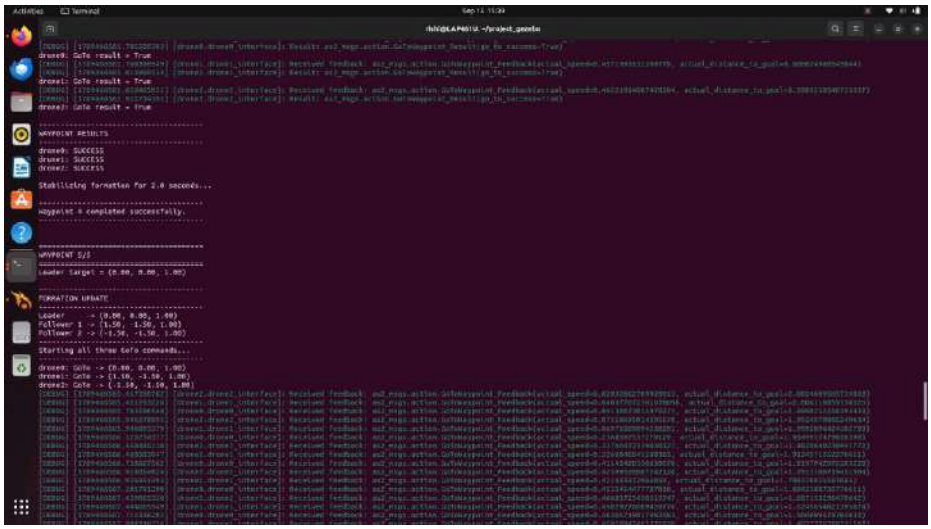


*Figure 11. Waypoint 4 and Waypoint 5 execution with successful GoTo results for all three drones.*

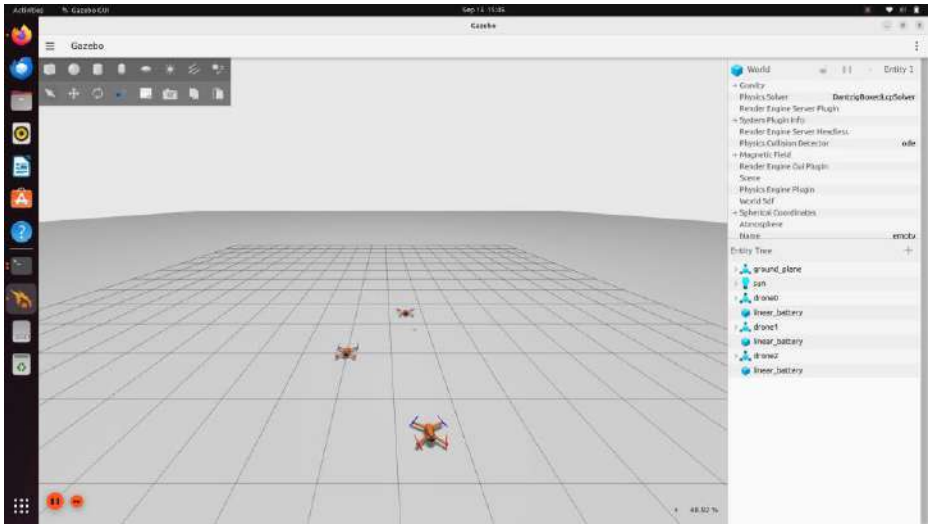


*Figure 12. Gazebo 3D view of the three drones flying in formation during mission execution.*

## 19. GoTo Feedback

During execution, AeroStack2 published GoToWaypoint action feedback for each drone, including actual_speed and actual_distance_to_goal. This feedback was visible in the terminal output throughout each waypoint transit and provides direct evidence that each drone was actively navigating toward its computed target, rather than the GoTo call simply returning a static success value.

No final position-accuracy figure (e.g. steady-state error in meters) was captured during this run, so no such number is reported here; only the qualitative evidence of in-transit feedback and the final True/SUCCESS results are claimed.

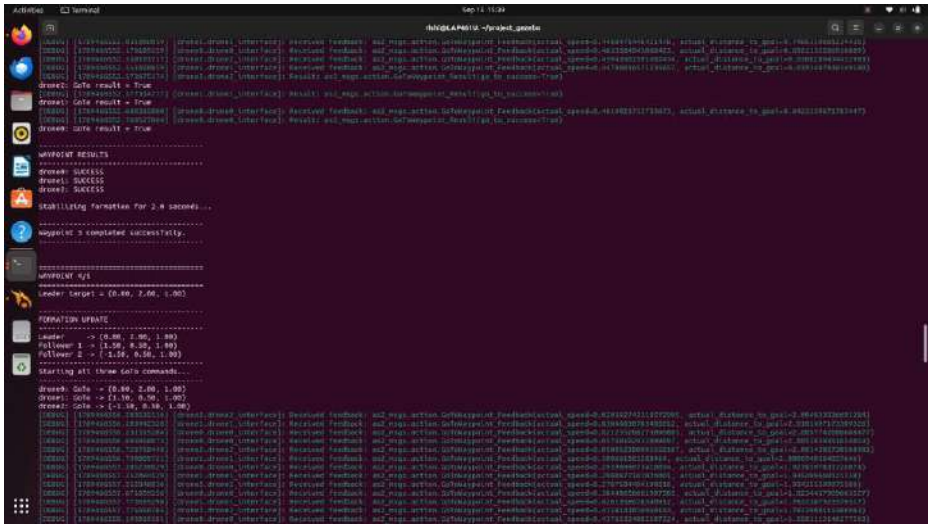


*Figure 13. AeroStack2 GoToWaypoint action feedback showing actual speed and actual distance-to-goal during formation execution.*

## 20. Mission Completion

```
Waypoint 5 completed successfully.

FORMATION MISSION COMPLETED SUCCESSFULLY
```

This confirms that the leader completed its full closed route — (0,0) → (2,0) → (2,2) → (0,2) → (0,0) — and that both followers tracked the corresponding offset targets at every waypoint along the way, with the formation returning to its starting configuration before landing.

## Landing Results

```
LANDING RESULTS
drone0: SUCCESS
drone1: SUCCESS
drone2: SUCCESS

All landing operations completed.
```

All three drones landed successfully following mission completion, confirming that the complete mission lifecycle — initialization, arming, Offboard, takeoff, formation navigation, and landing — was executed end-to-end without manual intervention beyond the three ENTER-key checkpoints in the script.

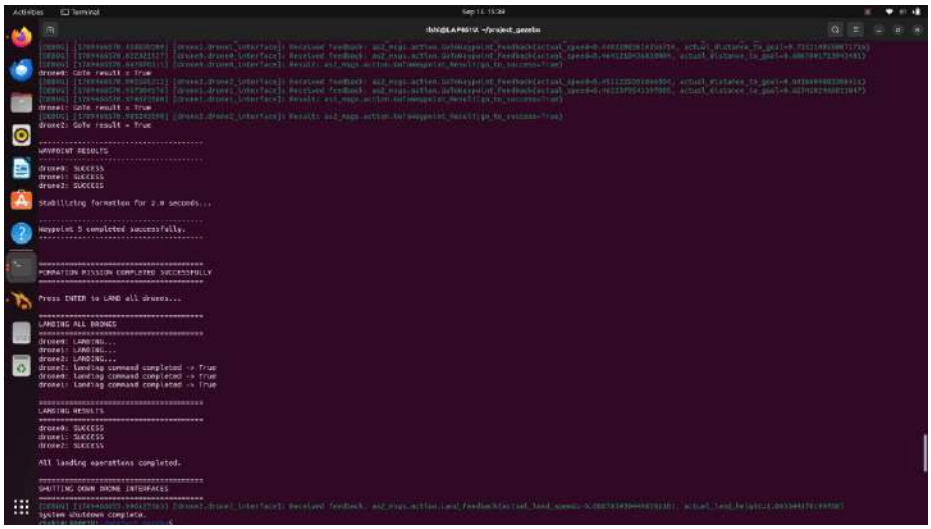


*Figure 14. Mission completion, successful landing of all three drones, and drone-interface shutdown.*

## Final Platform Verification

```
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash

for d in drone0 drone1 drone2; do
  echo "========== $d =========="
  ros2 topic echo /$d/platform/info --once
done
```

| Drone | Connected | Armed | Offboard | Status state |
|---|---|---|---|---|
| drone0 | true | false | true | 0 |
| drone1 | true | false | true | 0 |
| drone2 | true | false | true | 0 |

*Table 9. Final /platform/info state, captured after landing and shutdown.*

The captured final state confirms that all three drones remained connected and were disarmed (armed = false) after landing. The offboard field was still reported as true in this captured output. This is recorded here as an observed simulator/interface state; it is not treated as a mission failure, because the landing commands themselves returned SUCCESS for all three drones, and no separate offboard-disable step was invoked after landing in this version of the script.

## Validation and Results

| Validation category | Result |
|---|---|
| Interface connectivity (drone0/1/2) | Connected = true for all three drones |
| Arming | arm result = True for all three drones |
| Offboard activation | offboard result = True for all three drones |
| Takeoff | SUCCESS for all three drones, altitude 1.0 m reached |
| Waypoint execution (WP1–WP5) | GoTo result = True for all three drones at every waypoint |
| Formation coordination | Followers tracked leader-derived offsets at each waypoint |
| Return to initial waypoint | WP5 = WP1, formation returned to start configuration |
| Mission completion | “FORMATION MISSION COMPLETED SUCCESSFULLY” printed |
| Landing | SUCCESS for all three drones |
| Final platform verification | connected = true, armed = false for all three drones |

*Table 10. Summary of validation checks performed and their results.*

## Technical Analysis

The successful run demonstrates several distinct technical capabilities working together:

### *Multi-agent initialization and ROS 2 namespaces*

Three independent AeroStack2 DroneInterface objects were instantiated under separate ROS 2 namespaces (/drone0, /drone1, /drone2) within a single Python process, and all three reported connected = true prior to any flight command.

### *Sequential preparation and concurrent command execution*

Takeoff, GoTo and landing were dispatched using one Python thread per drone, and the threads were joined before the mission advanced to the next stage. Arming and Offboard activation were performed sequentially during the preparation stage. This allows the three drones to begin each concurrent motion stage within the same command window, supporting synchronized waypoint execution.

### *Relative formation geometry and waypoint-based coordination*

Follower targets are derived deterministically from the leader's planned waypoint using fixed offsets, so the same formation shape is reproduced at every one of the five waypoints without needing a separate formation-tracking loop.

### *Action feedback and mission sequencing*

GoToWaypoint feedback (actual_speed, actual_distance_to_goal) confirms that each GoTo call resulted in genuine motion rather than an instantaneous state change, and the mission sequencing (arm → offboard → takeoff → five waypoints → land → shutdown) matches the intended mission design exactly.

### *Safe landing and post-mission verification*

Landing was issued concurrently and confirmed SUCCESS for all three drones, and a separate, independently-run /platform/info check was used to verify the final system state rather than relying solely on the mission script's own reported results.

.

## OVERVIEW

- Structuring a multi-agent ROS 2 system using per-drone namespaces and a single coordinating Python process.
- Using the AeroStack2 DroneInterface abstraction (arm, offboard, takeoff, go_to, land) for simulation-based multi-drone development.
- Coordinating multiple drones concurrently using Python threading, and the importance of joining threads before advancing mission stages.
- Debugging an API argument-ordering issue by tracing a call through to the correct submodule method (go_to.go_to() vs. go_to()).
- Reading and interpreting ROS 2 action feedback (actual_speed, actual_distance_to_goal) as evidence of in-progress motion.
- Designing a mission sequence (arm → offboard → takeoff → waypoints → land → shutdown) that is both reproducible and verifiable.
- Collecting independent verification evidence (platform-info checks, node/topic listings) rather than relying only on a script's self-reported output.

## Appendix

### Appendix A — Complete Source Code

The complete, unmodified source code for leader_follower_waypoint_backup.py is provided in full in Section 15 (Complete Working Source Code) and is not duplicated here.

### Appendix B — Important Terminal Commands

```
# Environment
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
cd ~/project_gazebo

# Launch / stop
./launch_as2.bash -m
./launch_ground_station.bash -m -v
./stop.bash

# Run the mission
python3 leader_follower_waypoint_backup.py

# Platform verification
```

```
for d in drone0 drone1 drone2; do
  echo "========== $d =========="
  ros2 topic echo /$d/platform/info --once
done

# Supplementary ROS 2 interface checks
ros2 node list | grep -E '^/drone(0|1|2)/' | grep -E \
  'controller|motion|platform|state_estimator|GoTo|Takeoff'
ros2 topic list | grep -E '^/drone(0|1|2)/motion_reference'
ros2 action list | grep -E '^/drone(0|1|2)/' | grep -Ei 'goto|takeoff|land'
```

## Appendix C — Mission Waypoint Table

| WP | Leader | Follower 1 | Follower 2 |
|---|---|---|---|
| 1 | (0.00, 0.00, 1.00) | (1.50, -1.50, 1.00) | (-1.50, -1.50, 1.00) |
| 2 | (2.00, 0.00, 1.00) | (3.50, -1.50, 1.00) | (0.50, -1.50, 1.00) |
| 3 | (2.00, 2.00, 1.00) | (3.50, 0.50, 1.00) | (0.50, 0.50, 1.00) |
| 4 | (0.00, 2.00, 1.00) | (1.50, 0.50, 1.00) | (-1.50, 0.50, 1.00) |
| 5 | (0.00, 0.00, 1.00) | (1.50, -1.50, 1.00) | (-1.50, -1.50, 1.00) |

*Table 11. Mission waypoint table.*

## Appendix D — Final Validation Output

```
========== drone0 ==========
connected: true
armed: false
offboard: true
status.state: 0
========== drone1 ==========
connected: true
armed: false
offboard: true
status.state: 0
========== drone2 ==========
connected: true
armed: false
offboard: true
status.state: 0
```

*This is the final /platform/info output captured after landing and shutdown, corresponding to Table 9.*